

 RNN_26 (1).ipynb

Act as an expert Senior Python & Machine Learning Instructor. I have attached a Jupyter Notebook (`.ipynb`)

I do not want high-level summaries or generic documentation. Instead, please provide an exhaustive, line-by-line, mechanical breakdown of every single operational line of code in this file.

For EVERY line of code (especially the mathematical steps, array manipulations, and plotting configurations), please address the following structural points:

1. UNDERLYING MECHANICS & ARGUMENTS: What do the specific function parameters or arguments mean at a low level? (For example, if a line reads `axis=0`, `full_matrices=False`, or `::-1`, explain exactly what that specific configuration tells the computer to do step-by-step).
2. STRUCTURAL & SHAPE TRANSFORMATION: What are the input array shapes and data types before this line executes, and what is the exact output shape/data type immediately after it executes? (Explicitly use matrix dimensions like (N, D), (D, D), etc., to show how data is being stretched, collapsed, or transformed).
3. BEHAVIORAL ALTERNATIVES: What would happen to the code's behavior, execution flow, data shapes, and logic if those specific parameters were changed to other valid/invalid states? (Provide counter-examples of alternative parameter values and explain why they would either alter the outcome or break the script).
4. ALGORITHMIC & MATH JUSTIFICATION: Why is this exact configuration mathematically or structurally mandatory for the algorithm to function properly?

Please maintain this deep, parameter-focused engineering perspective throughout your entire line-by-line response.

Cell 1: Dependency Imports

python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf

from sklearn.preprocessing import MinMaxScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout
```

Line 1: `import numpy as np`

- 1. Underlying Mechanics & Arguments:** This statement loads the NumPy (Numerical Python) library into memory and assigns it the namespace alias `np`. Mechanically, Python searches `sys.path` for the compiled NumPy binaries (`.so` / `.pyd`) and source files, initializes the module object, and registers it in `sys.modules`. This provides access to highly optimized, C-implemented vectorization and contiguous memory block arrays (`ndarray`).
- 2. Structural & Shape Transformation:** No structural multi-dimensional array transformations occur. The statement introduces the `np` module object into the local symbol table.
- 3. Behavioral Alternatives:** Omitting `as np` forces the explicit use of `numpy.array()`. Changing it to an invalid identifier like `import numpy as 123` raises a `SyntaxError`. Importing sub-modules explicitly (e.g., `from numpy import array`) can pollute the local namespace and hide the origin of structural array operations.
- 4. Algorithmic & Math Justification:** Machine learning workloads depend on element-wise arithmetic and rapid linear algebra computations. NumPy arrays use native, contiguous memory layouts that sidestep the type-checking and pointer-indirection overhead of Python lists. This layout allows the CPU to leverage SIMD (Single Instruction, Multiple Data) instruction sets.

Line 2: `import pandas as pd`

- 1. Underlying Mechanics & Arguments:** This line loads the Pandas library and binds it to the alias `pd`. Pandas is engineered on top of NumPy, packaging its arrays into `Series` (1D labeled arrays) and `DataFrames` (2D tabular data structures with heterogeneous column types, integrated alignment, and axis labeling).

2. **Structural & Shape Transformation:** No mathematical structures are altered; the `pd` reference is injected into the runtime environment's namespace.
3. **BehaviorAL Alternatives:** Avoiding the alias requires calling `pandas.DataFrame`. Replacing it with a conflicting name like `import pandas as np` overwrites the NumPy reference, triggering an `AttributeError` when attempting to call numpy-specific methods later in the execution flow.
4. **Algorithmic & Math Justification:** Financial time-series data requires relational alignment, chronological indexing, and column-wise sub-selection. Pandas structures this metadata, ensuring that sequential operations match the correct data points before passing raw arrays to deep learning pipelines.

Line 3: `import matplotlib.pyplot as plt`

1. **Underlying Mechanics & Arguments:** This statement imports the state-based plotting interface `pyplot` from the `matplotlib` package, aliasing it as `plt`. Mechanically, it constructs an underlying engine capable of interacting with system graphic backends (e.g., TkAgg, Inline/Ipypthon) to convert digital arrays into rasterized or vectorized graphics.
2. **Structural & Shape Transformation:** No direct tensor modifications occur. It loads visualization classes and global state tracking objects into memory.
3. **BehaviorAL Alternatives:** Without an active backend handler (such as in headless server environments), executing plotting configurations without setting `matplotlib.use('Agg')` can crash the runtime with a connection or display error.
4. **Algorithmic & Math Justification:** Evaluating deep learning predictions visually requires rendering continuous true values alongside model outputs. This conversion maps abstract multi-dimensional scalar structures onto bounded 2D pixel coordinate spaces.

Line 4: `import yfinance as yf`

1. **Underlying Mechanics & Arguments:** This line imports the `yfinance` library as `yf`. It acts as an open-source scraper that wraps Yahoo Finance's public API endpoints, abstracting HTTP GET requests and converting received JSON payloads directly into structured Pandas DataFrames.
2. **Structural & Shape Transformation:** Modifies the global environment by instantiating the `yf` module.
3. **BehaviorAL Alternatives:** If the system lacks an active network socket or the endpoint changes its response schema, this module will raise an unhandled connection or parsing exception during API inquiries.
4. **Algorithmic & Math Justification:** Deep learning models require external, standardized historical datasets. This wrapper automates data acquisition, ensuring that raw market information is delivered cleanly to the ingestion pipeline.

Line 5: `from sklearn.preprocessing import MinMaxScaler`

1. **Underlying Mechanics & Arguments:** This statement extracts the specific class `MinMaxScaler` from `sklearn.preprocessing`. Rather than loading the entire `scikit-learn` ecosystem, it precisely targets the class responsible for bounding values into a uniform range.
2. **Structural & Shape Transformation:** Extracted class reference is appended to the local namespace.
3. **BehaviorAL Alternatives:** Importing the root package `import sklearn` forces long calls like `sklearn.preprocessing.MinMaxScaler()`. Using an incorrect string path raises a `ModuleNotFoundError`.
4. **Algorithmic & Math Justification:** Neural networks use gradient-based optimization algorithms. Raw financial data often exhibits high variance and extreme scales, which can cause exploding gradients or saturation in non-linear activation functions. Restricting features to a strict interval standardizes the optimization surface.

Line 6: `from tensorflow.keras.models import Sequential`

1. **Underlying Mechanics & Arguments:** This imports the Keras `Sequential` API class from the TensorFlow backend. Mechanically, it prepares a linear stack container where tensors flow sequentially through layers from input to output, tracking computational operations within a directed acyclic graph.
2. **Structural & Shape Transformation:** Registers the model constructor class in the execution context.
3. **BehaviorAL Alternatives:** The alternative is Keras's Functional API (`from tensorflow.keras import Model`). Using `Sequential` prevents the design of non-linear structural variations, such as residual skip-connections or multi-branch inputs.
4. **Algorithmic & Math Justification:** Standard forward-propagation models feed outputs directly into subsequent layers. The `Sequential` container formalizes this execution pattern, enforcing automated shape propagation through every registered matrix operation.

Line 7: `from tensorflow.keras.layers import Dense, LSTM, Dropout`

1. **Underlying Mechanics & Arguments:** This statement explicitly imports three core layer objects: `Dense` (fully-connected linear mapping layer), `LSTM` (Long Short-Term Memory recurrent layer), and `Dropout` (stochastic regularization layer).
2. **Structural & Shape Transformation:** Imports the blueprints for these structural components without instantiating objects or defining matrix dimensions.
3. **BehaviorAL Alternatives:** Using alternative layer definitions, such as `SimpleRNN` or `GRU`, completely changes the mathematical operations applied to passing tensors, altering how hidden states are updated.
4. **Algorithmic & Math Justification:** Deep learning architectures are constructed by stacking distinct mathematical operations. These specific imports provide the mathematical building

blocks required to handle recurrent relationships, execute linear projections, and apply regularization.

Cell 2: Data Acquisition & Preprocessing

python

```
ticker = 'GOOGL'
data = yf.download(ticker, start='2015-01-01', end='2023-12-31')

dataset = data[['Close']].values

train_size = int(len(dataset) * 0.8)

train_data = dataset[:train_size]
test_data = dataset[train_size:]

scaler = MinMaxScaler(feature_range=(0, 1))
scaled_train = scaler.fit_transform(train_data)
scaled_test = scaler.transform(test_data)
```

Line 1: `ticker = 'GOOGL'`

- Underlying Mechanics & Arguments:** Instantiates a string object containing `'GOOGL'` and binds it to the variable name `ticker` within the local scope.
- Structural & Shape Transformation:** Generates a primitive string structure. Shape: Scalar / Primitive string.
- BehaviorAL Alternatives:** Providing an invalid ticker name (e.g., `'INVALID_XYZ'`) passes a syntax-valid but logically broken string to the API, returning an empty DataFrame or a download error.
- Algorithmic & Math Justification:** Establishes a variable to dynamically query remote asset databases without hardcoding identifiers across downstream execution blocks.

Line 2: `data = yf.download(ticker, start='2015-01-01', end='2023-12-31')`

- Underlying Mechanics & Arguments:** Invokes the `download` method of the `yfinance` module. Arguments pass the asset identifier string, an inclusive start date string, and an exclusive end date string. The backend builds a query URL, executes an HTTP request, receives historical stock data, parses the payload, and instantiates a Pandas DataFrame.
- Structural & Shape Transformation:** Input: String arguments. Output: A 2D Pandas DataFrame named `data` containing stock metrics. Assuming approximately 252 trading days per year over

9 years, the resulting matrix shape is roughly $(N_{days}, 6)$, where columns represent open, high, low, close, adjusted close, and volume values.

3. **BehaviorAL Alternatives:** Setting `start` later than `end` returns an empty DataFrame or raises an error. Omitting the dates defaults the download to maximum historical data, dramatically expanding the matrix dimensions and changing subsequent dataset indices.
4. **Algorithmic & Math Justification:** Deep time-series networks require continuous historical logs. Bounding the data window explicitly defines the sample space, ensuring reproducible results across training iterations.

Line 3: `dataset = data[['Close']].values`

1. **Underlying Mechanics & Arguments:** This multi-step extraction isolates the `'Close'` column of the DataFrame using double brackets to ensure the output remains a 2D tabular slice, rather than a 1D series. The `.values` attribute accesses the underlying NumPy array, bypassing Pandas metadata wrappers.
2. **Structural & Shape Transformation:** Input: 2D Pandas DataFrame of shape $(N, 6)$. Output: A 2D NumPy array (`ndarray`) of shape $(N, 1)$ containing float64 values.
3. **BehaviorAL Alternatives:** Using single brackets `data['Close'].values` drops a dimension, producing a 1D shape $(N,)$ instead of a 2D shape $(N, 1)$. This layout mismatch will break Scikit-Learn transformers, which expect explicit matrix axes.
4. **Algorithmic & Math Justification:** Recurrent Neural Networks designed for time-series forecasting learn mapping functions from target histories. Extracting the closing price isolates the single variable under analysis, while converting it to a raw NumPy array strips away Pandas overhead for faster matrix operations.

Line 4: `train_size = int(len(dataset) * 0.8)`

1. **Underlying Mechanics & Arguments:** Evaluates the total length of the `dataset` array along its leading axis, multiplies that integer by `0.8`, and passes the resulting float to the `int()` constructor, which truncates any decimal component.
2. **Structural & Shape Transformation:** Input: 2D array of shape $(N, 1)$. Output: A scalar Python integer representing 80% of the row count.
3. **BehaviorAL Alternatives:** Omitting the `int()` truncation leaves a float value, which will cause a `TypeError` when used later as an array index slice.
4. **Algorithmic & Math Justification:** Time-series models must be validated on data that succeeds the training window to prevent lookahead bias. Splitting the rows sequentially ensures that the model learns from historical sequences and tests on future data.

Line 5: `train_data = dataset[:train_size]`

1. **Underlying Mechanics & Arguments:** Slice-extracts rows from index `0` up to (but excluding) `train_size` along the first axis of the array, leaving the second axis intact.
2. **Structural & Shape Transformation:** Input: Entire array of shape $(N, 1)$. Output: A 2D slice array named `train_data` with an exact shape of $(N_{train}, 1)$, where $N_{train} = \lfloor N \times 0.8 \rfloor$.
3. **BehaviorAL Alternatives:** Modifying the slice syntax to randomly shuffle indices (e.g., using `train_test_split`) destroys the sequential continuity of the data, invalidating the recurrent learning objective.
4. **Algorithmic & Math Justification:** Retaining chronological order is critical for time-series models. This slice creates a clean training window consisting entirely of historical observations.

Line 6: `test_data = dataset[train_size:]`

1. **Underlying Mechanics & Arguments:** Slices the remaining portion of the array, starting from index `train_size` and extending to the final row.
2. **Structural & Shape Transformation:** Input: Array of shape $(N, 1)$. Output: A separate 2D slice array named `test_data` with an exact shape of $(N_{test}, 1)$, where $N_{test} = N - N_{train}$.
3. **BehaviorAL Alternatives:** If an overlap occurs (e.g., `dataset[train_size-1:]`), the final row of the training set leaks into the test set, creating a slight data validation defect.
4. **Algorithmic & Math Justification:** Isolates an independent testing dataset that occurs chronologically after the training data. This mirrors a production environment, allowing you to test how well the model generalizes to future market behavior.

Line 7: `scaler = MinMaxScaler(feature_range=(0, 1))`

1. **Underlying Mechanics & Arguments:** Instantiates an instance of the `MinMaxScaler` object. The `feature_range` argument accepts a tuple specifying the targeted lower and upper boundaries for scaling. Mechanically, it configures internal scaling equations:

$$\bar{x} = \frac{x - x_{min}}{x_{max} - x_{min}} \times (max - min) + min$$

2. **Structural & Shape Transformation:** Input: Configuration tuple. Output: An un-initialized transformer object tracking scaling hyper-parameters in memory.
3. **BehaviorAL Alternatives:** Expanding the range (e.g., `(-10, 10)`) alters the input distributions, which can cause model saturation depending on the activation functions used.
4. **Algorithmic & Math Justification:** Restricting inputs to a strict $[0, 1]$ interval normalizes variance across the dataset. This keeps activation layers working within their most responsive gradient regions, accelerating model convergence.

Line 8: `scaled_train = scaler.fit_transform(train_data)`

1. **Underlying Mechanics & Arguments:** Invokes the `fit_transform` method on the `train_data` array. Mechanically, the `fit` step scans the training rows to locate the absolute minimum (x_{min}) and maximum (x_{max}) values. The `transform` step applies the normalization equation element-wise across the entire array.
2. **Structural & Shape Transformation:** Input: A raw array of shape $(N_{train}, 1)$. Output: A scaled array of shape $(N_{train}, 1)$ with data type forced to float64, where every element is bounded between 0.0 and 1.0.
3. **BehaviorAL Alternatives:** If you execute `fit_transform` on the entire dataset before splitting, the minimum and maximum values of the test data leak into the training process. This introduces data leakage, leading to overly optimistic validation metrics.
4. **Algorithmic & Math Justification:** The scaling parameters must be calculated exclusively from the training partition to ensure that the test partition remains completely unseen, preserving the integrity of your validation metrics.

Line 9: `scaled_test = scaler.transform(test_data)`

1. **Underlying Mechanics & Arguments:** Calls the `transform` method on `test_data` using the min/max parameters previously extracted from `train_data`. It does *not* recalculate new scaling metrics for the test set.
2. **Structural & Shape Transformation:** Input: Test array of shape $(N_{test}, 1)$. Output: Normalized test array of shape $(N_{test}, 1)$. Note that individual values can land outside the $[0, 1]$ range if the test set contains prices higher or lower than those seen in the training data.
3. **BehaviorAL Alternatives:** Running `.fit_transform(test_data)` here recalculates a local minimum and maximum from the test set itself. This shifts the underlying distribution relative to what the model learned during training, breaking the consistency of your inputs and ruining downstream predictions.
4. **Algorithmic & Math Justification:** To maintain mathematical consistency, the test data must be scaled using the exact same transformation function as the training data. This ensures that a given scaled value represents the exact same absolute price across both sets.

Cell 3: Sequenced Tensor Transformation

python

```
def create_sequences(data, window_size=60):
    X, y = [], []
    for i in range(window_size, len(data)):
        X.append(data[i - window_size:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)
```

```

window_size = 50

X_train, y_train = create_sequences(scaled_train, window_size)
X_test, y_test = create_sequences(scaled_test, window_size)

X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

```

Lines 1–7: Sequence Generation Function

python

```

def create_sequences(data, window_size=60):
    X, y = [], []
    for i in range(window_size, len(data)):
        X.append(data[i - window_size:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)

```

- 1. Underlying Mechanics & Arguments:** Declares a sequence formatting function. It iterates from index `window_size` to the end of the input array. In each iteration, it extracts a continuous block of historical rows (`i - window_size` up to `i`) and stores it in list `X` , while appending the single value at index `i` (the target price) to list `y` . Finally, `np.array()` converts these lists into dense, contiguous memory blocks.
- 2. Structural & Shape Transformation:** Input: A 2D array of shape $(M, 1)$. Output: A tuple containing two distinct structures: a 2D array `X` of shape $(M - \text{window_size}, \text{window_size})$ and a 1D target array `y` of shape $(M - \text{window_size},)$.
- 3. BehaviorAL Alternatives:** Modifying the slice expression to `data[i - window_size:i+1, 0]` accidentally leaks the target price directly into the input feature vector. This makes the model look highly accurate during training, but completely useless in production.
- 4. Algorithmic & Math Justification:** Recurrent architectures process inputs over explicit temporal horizons. This function transforms standard tabular rows into sliding lookback windows, mapping historical contexts directly to their subsequent target outcomes.

Line 8: `window_size = 50`

- 1. Underlying Mechanics & Arguments:** Assigns the integer `50` to the variable `window_size` , defining the temporal lookback horizon for the sequence generator.
- 2. Structural & Shape Transformation:** Instantiates an integer scalar.
- 3. BehaviorAL Alternatives:** Setting the window to an extremely small value (e.g., `1`) strips away the sequential context, turning the architecture into a standard lagging regression network.

Conversely, setting it larger than the total sample size will prevent the loop from executing, resulting in empty arrays.

4. **Algorithmic & Math Justification:** Defines the memory boundaries for the network. The model will look back exactly 50 trading days into the past to identify the patterns it uses to predict the next day's price.

Line 9: `X_train, y_train = create_sequences(scaled_train, window_size)`

1. **Underlying Mechanics & Arguments:** Passes the normalized training array and the lookback window size (50) to the sequence generator function. Note that while the function definition defaults to a window size of 60 , this call explicitly overrides it with 50 .
2. **Structural & Shape Transformation:** Input: Array `scaled_train` of shape $(N_{train}, 1)$. Output: `X_train` becomes a 2D matrix of shape $(N_{train} - 50, 50)$, and `y_train` becomes a 1D array of shape $(N_{train} - 50,)$.
3. **BehaviorAL Alternatives:** Passing unscaled data here runs the sequence transformation successfully, but passes massive variance variations to downstream layers, which can destabilize backpropagation.
4. **Algorithmic & Math Justification:** Breaks the continuous historical training data into individual, overlapping training samples. This pairs each 50-day sequence of past prices with its corresponding next-day target price.

Line 10: `X_test, y_test = create_sequences(scaled_test, window_size)`

1. **Underlying Mechanics & Arguments:** Processes the normalized test array into historical sequences and target vectors using the same lookback window.
2. **Structural & Shape Transformation:** Input: Array `scaled_test` of shape $(N_{test}, 1)$. Output: `X_test` becomes a 2D matrix of shape $(N_{test} - 50, 50)$, and `y_test` becomes a 1D array of shape $(N_{test} - 50,)$.
3. **BehaviorAL Alternatives:** Using a different window size for the test set (e.g., 40) alters the matrix dimensions. This layout mismatch will throw a structural error when the model tries to process the data, as it expects input shapes to remain consistent across both partitions.
4. **Algorithmic & Math Justification:** Formats the test dataset into the exact same structural layout used during training. This ensures your validation metrics are computed over consistent lookback windows.

Line 11: `X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))`

1. **Underlying Mechanics & Arguments:** Invokes `np.reshape` to alter the dimensional layout of `X_train` . The target shape tuple preserves the existing row count (`shape[0]`) and lookback length (`shape[1]`), while adding a new trailing singleton dimension. This modification changes

how the array pointers navigate the underlying memory blocks without modifying the internal data points.

2. **Structural & Shape Transformation:** Input: 2D array of shape $(N_{train} - 50, 50)$. Output: 3D tensor layout of shape $(N_{train} - 50, 50, 1)$, with the final dimension explicitly representing a single feature channel.
3. **BehaviorAL Alternatives:** Omitting this transformation step causes a structural layout mismatch when passing data to recurrent layers. Keras RNN cells are hardcoded to expect a 3D input structure, so passing a 2D matrix will raise a structural exception.
4. **Algorithmic & Math Justification:** Keras recurrent layers require a 3D input format structured as (Samples, Timesteps, Features) . This reshape step explicit adds the third axis, signaling to the network that it is processing a single feature over time.

Line 12: `X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))`

1. **Underlying Mechanics & Arguments:** Applies the same structural reshape to the test tensor, adding the trailing feature axis.
2. **Structural & Shape Transformation:** Input: 2D array of shape $(N_{test} - 50, 50)$. Output: 3D tensor layout of shape $(N_{test} - 50, 50, 1)$.
3. **BehaviorAL Alternatives:** Reshaping to an incorrect dimensional layout (e.g., $(1, X_test.shape[0], X_test.shape[1])$) completely scrambles how the data points map to batch slots and timesteps, leading to immediate calculation failures.
4. **Algorithmic & Math Justification:** Aligns the test dataset with the 3D tensor structure required by Keras RNN layers, ensuring smooth inference tracking across validation operations.

Cell 4: Model Architecture Construction & Compilation

python

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, SimpleRNN, Dropout

model = Sequential([
    SimpleRNN(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)),
    Dropout(0.2),

    SimpleRNN(units=50, return_sequences=False),
    Dropout(0.2),

    Dense(units=25),
    Dense(units=1)
])
```

```
model.compile(optimizer='adam', loss='mean_squared_error')

model.summary()
```

Lines 1-2: Redundant Import Logs

1. **Underlying Mechanics & Arguments:** Re-imports the core architecture components. This script swaps out the `LSTM` layer imported in Cell 1 for a `SimpleRNN` layer.
2. **Structural & Shape Transformation:** Updates the environment's class definitions, replacing the previous `LSTM` identifier with the `SimpleRNN` class blueprint.
3. **BehaviorAL Alternatives:** If you attempt to instantiate a layer class without importing it first, Python throws a `NameError` and halts execution.
4. **Algorithmic & Math Justification:** Brings the necessary class constructors into scope, allowing you to build your recurrent network layers.

Lines 4–13: Deep Recurrent Architecture Definition

python

```
model = Sequential([
    SimpleRNN(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)),
    Dropout(0.2),
    SimpleRNN(units=50, return_sequences=False),
    Dropout(0.2),
    Dense(units=25),
    Dense(units=1)
])
```

Layer 1: `SimpleRNN(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1))`

1. **Underlying Mechanics & Arguments:** Instantiates the first hidden layer of the model. The `units=50` argument sets the dimensionality of the internal hidden state vector (h_t). `return_sequences=True` configures the layer to output its hidden state at every timestep in the sequence, rather than just the final one. The `input_shape` parameter defines the expected shape of incoming sequences (50 timesteps, 1 feature). Mathematically, it evaluates the classical RNN activation equation for each step:
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$
2. **Structural & Shape Transformation:** Input Tensor Shape: `(None, 50, 1)` . Output Tensor Shape: `(None, 50, 50)` . (The `None` token acts as a dynamic placeholder for the incoming batch size).
3. **BehaviorAL Alternatives:** Setting `return_sequences=False` on this first layer forces it to return only a single vector from the final timestep, reducing the output shape to `(None, 50)` . This will

break the next recurrent layer in the stack, which expects a full 3D sequence of states.

4. **Algorithmic & Math Justification:** Stacking recurrent layers requires passing sequential data between them. By enabling `return_sequences=True`, this layer passes its complete structural history down the line, allowing the next layer to extract deeper temporal patterns.

Layer 2: Dropout(0.2)

1. **Underlying Mechanics & Arguments:** Instantiates a regularization layer. During training, the `0.2` argument randomly zeros out 20% of the tensor's activation components in each forward pass. This is drawn from a Bernoulli distribution and scales the remaining active components by $1/(1 - 0.2) = 1.25$ to maintain consistent total activation energy.
2. **Structural & Shape Transformation:** Input Tensor Shape: `(None, 50, 50)`. Output Tensor Shape: `(None, 50, 50)`. (Dimensions remain unchanged; individual internal data fields are stochastically zeroed out).
3. **BehaviorAL Alternatives:** Setting the dropout rate to an invalid value like `2.0` raises a value boundary error. Setting it to `0.0` disables regularization entirely, exposing the complex recurrent networks to overfitting on volatile financial patterns.
4. **Algorithmic & Math Justification:** Financial time-series data contains significant noise. Randomly breaking co-dependencies between units forces the network to learn robust features rather than memorizing individual training points, helping it generalize better to unseen data.

Layer 3: SimpleRNN(units=50, return_sequences=False)

1. **Underlying Mechanics & Arguments:** Instantiates the second recurrent layer, containing 50 hidden units. Because `return_sequences` is set to `False`, this layer only returns the final hidden state vector (h_T) computed at the end of the time horizon.
2. **Structural & Shape Transformation:** Input Tensor Shape: `(None, 50, 50)`. Output Tensor Shape: `(None, 50)`. (Collapses the temporal axis, transforming the sequence into a static feature representation).
3. **BehaviorAL Alternatives:** Setting `return_sequences=True` here passes a 3D tensor to the subsequent `Dense` layer. While modern Keras versions can handle this by applying the dense weights to each timestep individually, it alters the downstream architecture and changes your final output shape.
4. **Algorithmic & Math Justification:** Since the goal is to predict a single price value for the next trading day, the network needs to condense its temporal sequence down into a single summary vector before passing it to the final regression layers.

Layer 4: Dropout(0.2)

1. **Underlying Mechanics & Arguments:** Applies a second stochastically independent regularization mask, zeroing out 20% of the elements in the incoming summary vector.

2. **Structural & Shape Transformation:** Input Tensor Shape: (None, 50) . Output Tensor Shape: (None, 50) .
3. **BehaviorAL Alternatives:** Increasing this rate too high can starve the final projection layers of critical features, leading to underfitting and poor predictive performance.
4. **Algorithmic & Math Justification:** Adds a final layer of protection against overfitting, ensuring the condensed temporal representations remain robust before they are used to calculate the final prediction.

Layer 5: Dense(units=25)

1. **Underlying Mechanics & Arguments:** Instantiates a fully-connected layer that applies a linear transformation to the incoming data. It projects the 50-dimensional input vector down to 25 dimensions by performing a standard matrix multiplication followed by adding a bias vector:

$$y = X \cdot W_{dense1} + b_{dense1}$$

2. **Structural & Shape Transformation:** Input Tensor Shape: (None, 50) . Output Tensor Shape: (None, 25) .
3. **BehaviorAL Alternatives:** Introducing a non-linear activation function here (e.g., `activation='relu'`) can help capture more complex feature interactions, but it may also increase the risk of gradient saturation depending on your weight initializations.
4. **Algorithmic & Math Justification:** Acts as an intermediate step-down layer, condensing the temporal features into a tighter representation before making the final prediction.

Layer 6: Dense(units=1)

1. **Underlying Mechanics & Arguments:** Instantiates the final output layer. It performs a linear projection down to a single dimension to produce a continuous real value.
2. **Structural & Shape Transformation:** Input Tensor Shape: (None, 25) . Output Tensor Shape: (None, 1) .
3. **BehaviorAL Alternatives:** Accidentally adding a bounded activation function like `activation='sigmoid'` restricts the output exclusively to the range $[0, 1]$. If the target data is later inverted back to its original price scale, this bounding can cause severe distortion and ruin the predictions.
4. **Algorithmic & Math Justification:** Regression tasks require an unbounded linear output layer to freely predict the full spectrum of continuous real values.

Line 15: `model.compile(optimizer='adam', loss='mean_squared_error')`

1. **Underlying Mechanics & Arguments:** Configures the model's optimization engine. The `'adam'` argument initializes the Adaptive Moment Estimation optimizer, which dynamically adjusts individual learning rates for each parameter based on first (m_t) and second (v_t) moments of the

gradients. The 'mean_squared_error' argument establishes the optimization metric, minimizing the average squared difference between predictions and targets during training:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

2. **Structural & Shape Transformation:** Updates internal model state. The model compiled code is pushed to your hardware target (CPU or GPU), and weight tensors are prepared for update tracking.
3. **BehaviorAL Alternatives:** Swapping the loss function to 'categorical_crossentropy' breaks the training process immediately, as cross-entropy requires classification probability distributions rather than single continuous regression targets.
4. **Algorithmic & Math Justification:** The MSE loss function penalizes larger errors more heavily, making it an excellent fit for financial forecasting where minimizing large misses is critical. The Adam optimizer accelerates training by navigating complex optimization surfaces more efficiently than standard gradient descent.

Line 17: `model.summary()`

1. **Underlying Mechanics & Arguments:** Scans the compiled internal layer objects to compute their shape transitions and count the total number of trainable parameters. It prints this structured breakdown directly to stdout.
2. **Structural & Shape Transformation:** Has no effect on data structures or shapes; simply prints out structural information about the model.
3. **BehaviorAL Alternatives:** If invoked prior to compiling or defining your input shapes, it may raise an un-instantiated exception or fail to render parameter counts.
4. **Algorithmic & Math Justification:** Provides an absolute validation step to ensure your tensor shapes match expectations and parameters are flowing correctly across layers before you begin training.

Cell 5: Model Optimization Executions

python

```
history = model.fit(  
    X_train,  
    y_train,  
    batch_size=32,  
    epochs=10,  
    validation_split=0.1  
)
```

1. **Underlying Mechanics & Arguments:** Launches the model's training loop. It sets aside the final 10% of your training data for validation tracking (`validation_split=0.1`). The training data is split into mini-batches of 32 samples (`batch_size=32`), and the optimizer processes the entire training set 10 times (`epochs=10`). In each step, a batch of sequences flows forward through the network to calculate the loss, gradients are propagated backward via backpropagation through time (BPTT), and the Adam optimizer updates the internal weight matrices.
2. **Structural & Shape Transformation:** Input: Training tensors `X_train` ($N_{train_seq}, 50, 1$) and `y_train` ($N_{train_seq},$). During each epoch, the data is partitioned into training sequences of shape ($M_{fit}, 50, 1$) and validation sequences of shape ($M_{val}, 50, 1$). The function outputs a `History` object containing logs of the training and validation loss across iterations.
3. **BehaviorAL Alternatives:** Setting `batch_size=1` shifts the training process to pure stochastic gradient descent, drastically increasing training times and introducing high variance into gradient updates. Conversely, setting the batch size larger than the total sample size will throw an execution exception.
4. **Algorithmic & Math Justification:** Mini-batch gradient descent strikes a balance between training stability and computational efficiency. The validation split allows you to monitor how well the model generalizes to unseen data during training, providing an early warning sign if it begins to overfit the noise in your training set.

Cell 6: Evaluation Inference & Visualization

python

```
predictions = model.predict(X_test)
predictions = scaler.inverse_transform(predictions)

actual_prices = dataset[train_size + window_size:]

plt.figure(figsize=(14, 5))

plt.plot(actual_prices, color='black', label=f'Actual {ticker} Price')
plt.plot(predictions, color='green', label=f'Predicted {ticker} Price')

plt.title(f'{ticker} Stock Price Prediction')
plt.xlabel('Time')
plt.ylabel('Price')

plt.legend()
plt.show()
```

Line 1: `predictions = model.predict(X_test)`

1. **Underlying Mechanics & Arguments:** Disables dropout operations and executes a pure forward pass using the test features tensor `X_test`. Tensors pass through each layer to compute the final model outputs without calculating gradients or updating weights.
2. **Structural & Shape Transformation:** Input Tensor Shape: `(N_{test\_seq}, 50, 1)`. Output Tensor Shape: `(N_{test\_seq}, 1)` containing normalized values.
3. **BehaviorAL Alternatives:** Passing a single sequence instead of a 3D tensor will cause a structural error, as the model expects a full batch dimension.
4. **Algorithmic & Math Justification:** Generates the model's raw predictions for the test horizon, enabling a direct performance comparison against the true unseen prices.

Line 2: `predictions = scaler.inverse_transform(predictions)`

1. **Underlying Mechanics & Arguments:** Reverses the normalization scaling operation by passing the predictions through the inverse scaling function:

$$x_{raw} = \bar{x} \cdot (x_{max} - x_{min}) + x_{min}$$

2. **Structural & Shape Transformation:** Input Tensor Shape: Normalized array of shape `(N_{test\_seq}, 1)`. Output Tensor Shape: Denormalized array of shape `(N_{test\_seq}, 1)` restored to the original financial asset price scale.
3. **BehaviorAL Alternatives:** Skipping this inversion leaves your predictions on a relative scale between 0 and 1. Comparing these directly against actual stock prices in a chart or loss function will show massive, artificial errors.
4. **Algorithmic & Math Justification:** Performance evaluation metrics and visualizations must be computed on the data's original scale to provide accurate, meaningful insights into real-world performance.

Line 3: `actual_prices = dataset[train_size + window_size:]`

1. **Underlying Mechanics & Arguments:** Extracts the true unscaled closing prices that match the timeline of your test predictions. It offsets the starting index by adding `window_size` to `train_size`, account for the fact that the first 50 days of the test set are required to form the first prediction window.
2. **Structural & Shape Transformation:** Input: Base array of shape `(N, 1)`. Output: Slice array of shape `(N_{test} - 50, 1)`.
3. **BehaviorAL Alternatives:** Forgetting to include the `window_size` offset shifts your actual prices out of alignment with your predictions, rendering any downstream comparisons or visualizations inaccurate.
4. **Algorithmic & Math Justification:** Ensures your true targets and model predictions are perfectly aligned in time, allowing for a precise, day-by-day evaluation of the model's accuracy.

Line 4: `plt.figure(figsize=(14, 5))`

1. **Underlying Mechanics & Arguments:** Initializes a new Matplotlib figure canvas. The `figsize` argument accepts a tuple specifying the figure width (14 inches) and height (5 inches).
2. **Structural & Shape Transformation:** Instantiates a figure object and updates the plotting canvas layout.
3. **BehaviorAL Alternatives:** If you omit this configuration, Matplotlib defaults to a standard small box layout, which can compress and distort long time-series lines.
4. **Algorithmic & Math Justification:** Controls the dimensions of the output image, ensuring the chronological patterns remain clear and easy to analyze.

Line 5: `plt.plot(actual_prices, color='black', label=f'Actual {ticker} Price')`

1. **Underlying Mechanics & Arguments:** Binds the true price array to the canvas axis. The `color` parameter sets the rendered line to black, and the `label` parameter registers an identifier for the chart legend.
2. **Structural & Shape Transformation:** Converts data points into a continuous line graphic mapped to the canvas coordinates.
3. **BehaviorAL Alternatives:** Passing data with mismatched dimensions will raise a vector layout exception and halt the plot creation.
4. **Algorithmic & Math Justification:** Visualizes the ground-truth price history, establishing a clear baseline for evaluating the model's predictions.

Line 6: `plt.plot(predictions, color='green', label=f'Predicted {ticker} Price')`

1. **Underlying Mechanics & Arguments:** Overlays the model's denormalized predictions onto the same canvas as a green line.
2. **Structural & Shape Transformation:** Renders a second continuous line graphic onto the active plot canvas.
3. **BehaviorAL Alternatives:** If the predictions aren't correctly denormalized, this line will flatline near zero along the very bottom of the chart.
4. **Algorithmic & Math Justification:** Plots the predicted trends directly alongside the true prices, making it easy to visually assess how well the model tracked real-world market movements.

Lines 7–9: Chart Label Configurations

1. **Underlying Mechanics & Arguments:** Standardizes text values for the chart title, horizontal x-axis, and vertical y-axis.
2. **Structural & Shape Transformation:** Appends text descriptions to the chart canvas margins.

3. **BehaviorAL Alternatives:** Leaving these blank creates a barren, uninterpretable chart with no context around what asset or metrics are being displayed.
4. **Algorithmic & Math Justification:** Adds clear context and units to the visualization, turning abstract lines into an actionable performance report.

Line 11: `plt.legend()`

1. **Underlying Mechanics & Arguments:** Scans the active chart layers for registered label strings and constructs a diagnostic key inside the plot area.
2. **Structural & Shape Transformation:** Draws a label box onto the chart canvas.
3. **BehaviorAL Alternatives:** Skipping this step leaves the chart lines unlabeled, making it difficult to tell which line represents actual prices versus predictions.
4. **Algorithmic & Math Justification:** Clearly identifies each data series, ensuring the visualization is easy to read and analyze.

Line 12: `plt.show()`

1. **Underlying Mechanics & Arguments:** Instructs the underlying Matplotlib backend to compile all the active layers, render the complete graphic, and display it on screen. After rendering, it flushes the current figure state to reset the canvas for future plots.
2. **Structural & Shape Transformation:** Renders the final chart onto your display interface and clears the current plotting memory.
3. **BehaviorAL Alternatives:** In non-interactive environments, omitting this call can cause the script to finish without displaying the chart, or fail to save the output image.
4. **Algorithmic & Math Justification:** Finalizes the visualization process, outputting a clear performance chart to help you analyze your time-series model.